CLAUDE COMMUNITY WROCŁAW #4

20.10.2026

Trust, *but eval*

An agent you haven't evaluated is just a very confident guess.



Konrad *Dżbik*
Head of AI

```
– baseline
– grader
– budget
```

01 Agenda

~30 MIN + Q&A

01	**Poll and thesis**	Who can prove their skill actually helps?	~3.5 min
02	**Autonomy ladder**	Five levels, three questions on each	~2.5 min
03	**Demo #1: skill eval**	Skill vs bare prompt on 15 cases	~7 min
04	**Regression and grader**	Eval as CI for skills, LLM-as-judge	~5 min
05	**Demo #2: agent team**	/goal with a hard budget	~6.5 min
06	**Results and checklist**	Table, four questions, take-home repo	~5.5 min

After 30 minutes you can

→ Measure a skill against a baseline
→ Catch a silent regression in description
→ Stop an agent team with a hard budget in code

01 Quick poll

01

Who has a skill, plugin or agent setup in their repo?

Hands up.

02

Who can prove it actually helps?

...and say what it costs per run?

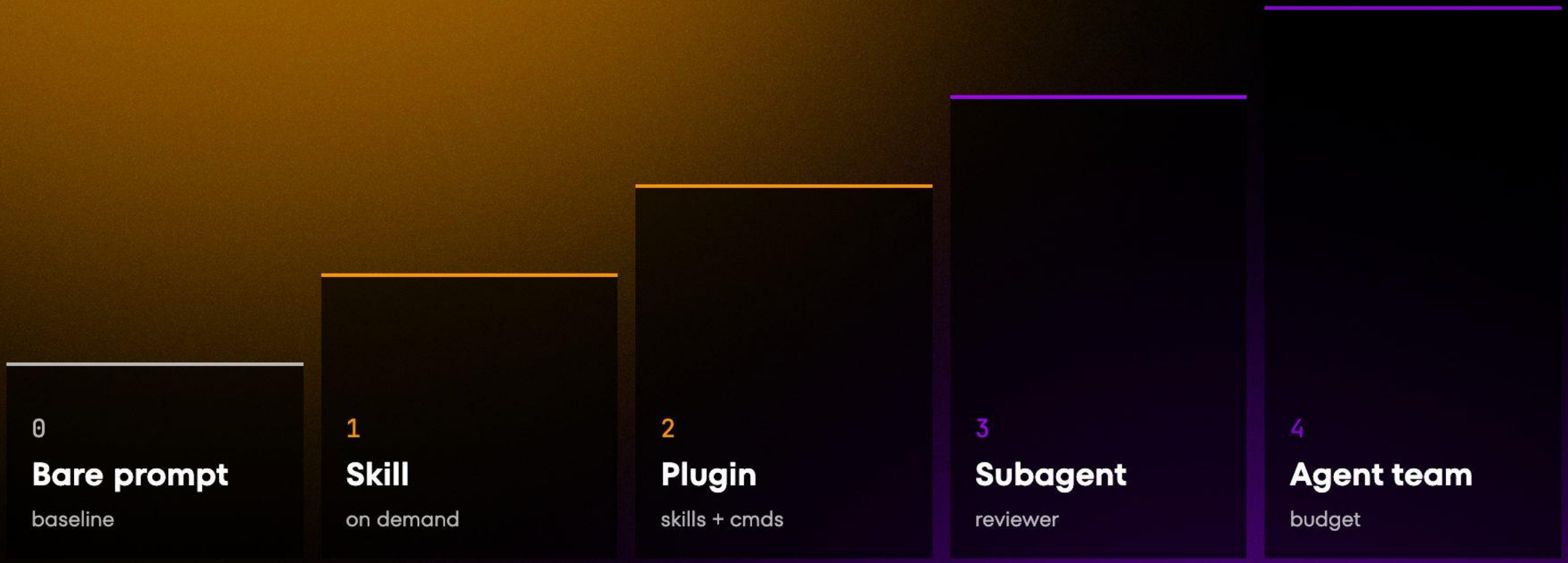
THE THESIS

Autonomy without evaluation is technical debt you repay in production.

Committed demo: on 2 of 15 cases the skill **loses** to the bare prompt (−0.29, −0.19). Without the eval you would never know.

02 The autonomy ladder

Every level adds autonomy, and one more thing you have to measure.



02 Three questions at every level

`baseline`

Is it better than the level below?

Same input, with and without. No baseline, no claim.

`grader`

Who decides what "better" means?

Assertions first, an LLM judge for the rest, calibrated on human labels.

`budget`

What does it cost?

Tokens, dollars and seconds per run, plus a stop condition.

Nobody merges a PR without tests. Why do we merge skills without evals?

03 Demo #1: one skill, 15 real cases

Skill under test

commit-message: Conventional Commits from a staged diff

Cases from real commits

vite and commitlint (MIT): 8 happy, 5 edge, 2 adversarial

Graders

regex for format, type, breaking, split · LLM judge for quality

evals/cases.jsonl

```
{"id": "adv-docs-typo-fix",
"input": {"diffs": ["ede50985fbe0.diff"]},
"expected": {"types": ["docs"]},
"tags": ["adversarial"]}

{"id": "edge-breaking-remove-option",
"expected": {"types": ["refactor","feat"],
"breaking": true},
"tags": ["edge", "breaking"]}
```

03 Live: skill vs baseline LIVE

```
$ claude plugin validate .

$ claude plugin eval . --case ci-remove-step

→ --ablation with-without is the default
→ each case: with skill + without skill, same prompt
→ Δ = with skill - without skill, per case

$ claude plugin eval . --ablation none --case ci-remove-step

→ after a one-line edit of the description: watch skill fired drop

$ git checkout -- skills/commit-message/SKILL.md
```

Grader: regex per case + Sonnet judge

Pass/fail: every assertion holds

~3 min for 15 cases × 3 runs × 2 arms

03 /skill-doctor: a report, not a lint

Usage and cost report

What each skill costs and how often it fires. Flag: never invoked. Requires Claude Code 2.1.252+.

Only the eval catches the description

`claude plugin validate` checks that frontmatter parses. A description that parses but no longer matches the prompt passes both.

`Same broken description, three checks`

`claude plugin validate` does the frontmatter parse	**pass**
`/skill-doctor` cost and how often it fires	**pass**
`claude plugin eval` does the skill fire on the prompt	**93% → 0%**

04 The grader becomes an agent

exact match → assertions → LLM judge

The judge sees what regex can't

Invented "why" in commit bodies, a defensible but wrong type. It passes 22% of skill runs where regex passes 64%.

Calibrate it: report agreement

10-case calibration sheet in the repo: label by hand, then report judge–human agreement. Judge ≠ model under test.

Case pattern: phased rollout

1. Phase 1: measure a baseline on a fixed task set
2. Gate: phase 2 only after beating that baseline
3. Same task set and graders in every phase

05 Demo #2: a team with a budget

ROLE

implementer

writes the code

→

ROLE

reviewer

approves or blocks the diff

→

ROLE

tester

runs tests, fails with a reason

```
$ claude --plugin-dir ..
> /trust-but-eval-demo:goal tight

$ python3 orchestrator.py # generous
--max-iterations 4 --token-budget 600000
$ python3 orchestrator.py # tight
--max-iterations 3 --token-budget 160000
```

Stop condition, checked before every call

Out of budget means stop and report. A hard stop, not a suggestion.

Checked by the orchestrator script, not by the LLM.

A tester FAIL goes back to the implementer with feedback.

05 Live: budget runs out

LIVE

```
$ python3 orchestrator.py --max-iterations 3 --token-budget 160000
[round 1] implementer DONE 28,869 tok | budget 18%
[round 1] reviewer APPROVE 15,759 tok | budget 28%
[round 1] tester FAIL 71,717 tok | budget 73%
[round 2] implementer KILLED 37,700 tok | budget 96%
STOPPED: budget exhausted · 154,045 / 160,000 tok · $0.21 · 56 s
```

(a) enough budget: success + cost report

(b) tight budget: stop + "done / left / next"

06 Before you ship the next agent

1. Do you have a baseline?
2. Do you have a grader and ≥10 cases?
3. Do you have a budget and a stop condition?
4. Would the level below be enough?

Take home today **trust-but-eval-demo** harness · skill · 15 cases `github.com/konraddzbik/cc-eval-workshops`

CLAUDE COMMUNITY WROCŁAW #4

Thank you!

Trust, *but eval.*

Questions? Preferably the uncomfortable ones.

Q&A

eval-explainer

Public repo — contributions welcome.

`github.com/CShark-Hub/eval-explainer`

CSHARK **Konrad** *Dżbik*